EF Core: Join Queries

Concise lesson summary — combining tables without loading full related entities, with key terms highlighted.

1. Core idea: Include() vs Join()

Include() loads related **entities** through navigation properties. **Join()** combines tables and lets you create a **custom result shape** containing only needed values.

Memory anchor: Need related entity objects? Think Include(). Need report/dashboard data? Think Join() + Select().

```
// Include: load related entities
var courses = context.Courses
    .Include(c => c.Instructor)
    .ToList();

// Join: return a custom report shape
var report = context.Courses
    .Join(context.Instructors,
        course => course.InstructorId,
        instructor => instructor.InstructorId,
        (course, instructor) => new
        {
            course.Title,
            course.Code,
            InstructorName =
                instructor.FirstName + " " + instructor.LastName
        })
    .OrderBy(x => x.Title)
    .ToList();
```

2. Join() syntax — four parts

```
Join(
    secondTable,
    firstKey,
    secondKey,
    resultShape)
```

• **secondTable**: the source to join with Courses (Instructors here).

• **firstKey**: key selector from the first source, e.g. course.InstructorId.

• **secondKey**: matching key selector from the second source, e.g. instructor.InstructorId.

• **resultShape**: defines what each matching pair returns.

Correct relationship: Course.InstructorId = Instructor.InstructorId. Joining unrelated keys can produce incorrect results.

3. Generated SQL idea

```
SELECT c.Title, c.Code,
       i.FirstName + N' ' + i.LastName
FROM Courses AS c
INNER JOIN Instructors AS i
    ON c.InstructorId = i.InstructorId
ORDER BY c.Title;
```

• This example uses **INNER JOIN**, so only courses with a matching instructor appear.

• **ToQueryString()** previews the SQL query shape; **ToList()** executes the query and materializes the report rows. Runtime logging can show executed SQL commands.

4. Why Join + Select for reports?

Approach	What it returns	Typical use
Include()	Course entities with related Instructor entities	Screens that need entity objects and navigation properties
Join() + Select	A custom result containing selected columns	Reports, exports, dashboards, search results, lightweight API responses

Why? Loading full entities when a report needs only a few fields can retrieve more data than necessary. Projection makes the result shape explicit.

5. Common mistakes

Avoid: loading full entities with Include() and then manually picking a few fields for a simple report. Consider a projection or Join query instead.

Avoid: joining unrelated keys such as CourseId and InstructorId. Join the actual relationship keys.

6. Joining more than two tables

The same idea can combine **Students + Enrollments + Courses** to create a report showing which student is enrolled in which course. Each join needs the correct matching keys.

Final takeaway

Include() → load related entities. Join() + Select → combine tables into a custom result containing only needed columns. Choose based on the result your application actually needs.

Note: the SQL shown is the lesson's example. Actual generated SQL depends on the LINQ query, model, provider, and EF Core version.